

Solving the capacitated vehicle routing problem with timing windows using rollouts and MAX-SAT

Harshad Khadilkar

1. TCS Research 2. IIT Bombay

Mumbai, India

harshad.khadilkar@tcs.com, harshadk@iitb.ac.in

Abstract—The vehicle routing problem is a well known class of NP-hard combinatorial optimisation problems in literature. Traditional solution methods involve either carefully designed heuristics, or time-consuming metaheuristics. Recent work in reinforcement learning has been a promising alternative approach, but has found it difficult to compete with traditional methods in terms of solution quality. This paper proposes a hybrid approach that combines reinforcement learning, policy rollouts, and a satisfiability solver to enable a tunable tradeoff between computation times and solution quality. Results on a popular public data set show that the algorithm is able to produce solutions closer to optimal levels than existing learning based approaches, and with shorter computation times than metaheuristics. The approach requires minimal design effort and is able to solve unseen problems of arbitrary scale without additional training. Furthermore, the methodology is generalisable to other combinatorial optimisation problems.

Index Terms—vehicle routing, policy rollouts, satisfiability

I. INTRODUCTION

The vehicle routing problem (VRP) involves the planning of journeys for a fleet of vehicles in order to serve a given set of customer demands. It is closely related to the travelling salesman problem (TSP), with the difference being that the TSP is solved by a single journey while the VRP can be solved by multiple vehicles with exclusive and exhaustive partitioning of the demand nodes. The goal of both TSP and VRP is to minimise the distance travelled (possibly in addition to number of vehicles utilised). A more realistic version of VRP is CVRP-TW, which includes a capacity constraint (C) on the total demand served by any given vehicle as well as time window (TW) constraints attached to each demand node. A precise definition of the problem is given in Section III.

CVRP-TW is highly commercially relevant, with applications in domains such as ride sharing, logistics, supply chain, and various service industries. A detailed overview of solution approaches is given in Section II. This paper addresses the following questions which appear to be incompletely answered by present literature:

- Can CVRP-TW be solved by a less design-intensive method than specialised rule-based systems?
- Can this method reduce the computational times of metaheuristic algorithms by utilising offline training?
- Can the methodology take advantage of known optimisation formulations of CVRP-TW?
- Can the approach be generic enough to be applied to other combinatorial optimisation problems?

The approach described in Sec. IV resolves these questions through a combination of reinforcement learning to enable policy based rollouts, and a satisfiability solver for optimisation of sub-problems produced by the first step. Experiments are reported on the popular data set shared by Solomon [1], [2]. While Monte Carlo Tree Search (MCTS) [3] offers a way of predicting future outcomes more reliably, the branching factor for realistic problem sizes makes it impractical for real-world use. Neural-MCTS [4] offers a way out of this dilemma, by allowing us to bias the policy rollouts based on a trained policy or value function.

II. RELATED WORK

Prior literature in this area is extensive, and can be broadly classified into four categories: heuristic or rule-based systems, meta-heuristics or randomised search, formal optimisation methods, and learning-based methods.

Heuristics have been worked on for many decades [5], [6]. Typically based on specially designed operators such as insertion [7] and 2-opt [8], [9]. They are the most competitive algorithms for the present problem, with near-optimal results on the Solomon data set [10], [11]. However, there is a lot of design effort involved.

Randomised search algorithms are very popular for vehicle routing. Comprehensive surveys [12], [13] reveal approaches such as Tabu search [14], [15], genetic algorithms [16], [17], and ant colony optimisation [18] among many other methods. However, these approaches tend to have long compute times. Prior literature [19], [20] shows that the computation times may be more than 100 times longer than those for heuristics or learning based approaches.

CVRP-TW is a combinatorial problem in the NP-hard class, but its formulation using mixed-integer linear programming is well-known [21], [22]. There are also variants in literature including robust formulations [23] which allow for uncertain travel times as well as online libraries for computing solutions [24]. However, the chief challenge in these approaches is the growth in computation times with the scale of the problem, and the unwieldy nature of big-M constraints [25] required for ordering of customers on routes.

In recent years, the majority of new algorithms use some form of learning approach, either through graph-inspired methods such as pointer networks [26], [27] and graph neural

networks [28], [29] or through reinforcement learning [30]–[33]. However, both sets of methods face significant challenges when incorporating constraints. Most studies tend to include vehicle capacity, but they either ignore time windows [30]–[32] or else incorporate them as soft constraints [33]. Furthermore, there do not appear to be any pure learning based approaches that are competitive with existing heuristics in terms of solution quality.

The goal of this paper is to provide a simple composable approach that incorporates the advantages of exact formulations, learning, and meta-heuristics. The algorithm proposed in Sec. III is generic enough to be applied to any combinatorial optimisation problem with a known formulation, and it can also be tuned to trade off the computation time of the solver and search components with the lack of exploration of greedy RL implementations.

III. PROBLEM DESCRIPTION

The capacitated vehicle routing problem with service time windows (CVRP-TW) assumes that a set of customers $\mathcal{C}$ is known, with their locations (x_i, y_i) , demanded load m_i , and service time windows $[T_{i,\min}, T_{i,\max}]$, where $i \in \mathcal{C}$, $x_i, y_i \in \mathbb{R}$, $T_{i,\min}, T_{i,\max} \in \mathbb{R}^+$, and $T_{i,\min} < T_{i,\max}$. In this paper, the vehicles start at a single depot o , and are able to travel between any two locations (fully connected graph), with the distance $d_{i,j}$ between any two customers $i, j \in \mathcal{C}$ given by the Euclidean metric. The solution approach described in the next section is directly applicable to the multi-depot case as well. We also have a fixed set of vehicles $\mathcal{V}$, the speed v of the vehicles (assumed constant on all edges and for all vehicles in the present work), and the maximum load M that any vehicle can carry in one trip. Then the objective of the problem [19] is to find the total distance J that minimises,

$$J = \min_{a_*, f_*, l_*} \left(\sum_{i,j,k} d_{i,j} a_{i,j,k} + \sum_{i,k} d_{o,i} f_{i,k} + \sum_{i,k} d_{o,i} l_{i,k} \right), \quad (1)$$

where $d_{i,j}$ is the distance from customer i to customer j , $d_{o,i}$ is the distance from origin (depot) to customer i , $a_{i,j,k}$ is an indicator variable which is 1 if vehicle k goes directly from customer i to customer j , $f_{i,k}$ is an indicator variable which is 1 if customer i is the first customer served by vehicle k , and $l_{i,k}$ is a similar indicator variable which is 1 if customer i is the last customer visited by vehicle k . Apart from constraints on $a_{i,j,k}$, $f_{i,k}$, and $l_{i,k}$ to take values from $\{0,1\}$, the other constraints are defined below in brief. A detailed description can be found in [19].

Every customer must be served within its specified time window. If $t_{i,k}$ is the time at which vehicle k visits i , then

$$T_{i,\min} \leq t_{i,k} \leq T_{i,\max}, \text{ if } f_{i,k} = 1 \text{ or } \exists j \text{ s.t. } a_{j,i,k} = 1 \quad (2)$$

The total load served by a vehicle must be at most equal to its capacity M . This is formalised as,

$$\sum_{i,j} m_j a_{i,j,k} + \sum_i m_i f_{i,k} \leq M \quad \forall k \quad (3)$$

Travel time constraints are applicable between any two locations, based on the distance between them, the speed v at which vehicles can travel, and the pre-specified service time Δ_j required at each customer.

$$t_{i,k} \geq \frac{d_{o,i}}{v} \text{ if } f_{i,k} = 1 \quad (4)$$

$$t_{i,k} \geq t_{j,k} + \Delta_j + \frac{d_{j,i}}{v} \text{ if } a_{j,i,k} = 1 \quad (5)$$

Clearly, this is a simplified version of a real-world situation where the number of vehicles also needs to be minimised, the vehicles can have different distance and velocity constraints, travel times can vary based on traffic conditions, vehicles can do multiple trips, in addition to other variations. The dynamic version of the problem will also allow customers to ‘pop up’ at arbitrary times, while a plan is being executed. However, the formulation described above is itself of interest because of its NP-hard nature, and the fact that exact solutions quickly become intractable as the problem size increases.

IV. METHODOLOGY

A. High-level description

An overview of several decades of work on this problem is given in Sec. II. One may generalise the outcomes as follows. Heuristics are the most competitive algorithms currently known in terms of solution quality and computation time, but require significant design effort. Furthermore, they cannot be generalised to other combinatorial optimisation problems. Meta-heuristics are similarly competitive, but have high computation times. Exact optimisation becomes rapidly infeasible as the problem scale increases. Finally, learning based solutions face challenges with respect to solution quality.

Consequently, the methodology proposed in this paper combines three different ideas into a single integrated approach as outlined in Algorithm 1. The meta-heuristic inspiration is used in the form of stochastic policy rollouts to identify good routing decisions. However instead of randomised local search, the rollouts are performed by a pre-trained reinforcement learning algorithm. The immediate forward path identified by rollout is optimised using a satisfiability solver in MAX-SAT mode. Finally, a post-processing MAX-SAT step further optimises the final routes. The individual components are described below.

B. Preprocessing of data set

Each data set first undergoes a preprocessing step when it is loaded, in order to characterise its properties. A simple procedure as described in Algorithm 2 separates all the customers into clusters, which are used for feature computation in Sec. IV-C. The algorithm is designed to work without predefining the number of clusters desired, as opposed to the typical k-means or related procedures [34]. Further characteristic quantities include a time threshold τ equal to the median travel time between all pairs of customers in the data set, the maximum time window t_{max} in the data set, and the largest inter-customer distance d_{max} in the data set. Each of these quantities is used in feature computation as described below.

Data: Customer data specification

Result: Vehicle routes and service times

Initialise: Single vehicle at depot, parameter κ ;

while *at least one customer yet to be served* **do**

identify further feasible customer-vehicle pairs;
shortlist top κ pairs identified by RL;
do stochastic rollouts using RL policy;
choose the decision with the lowest total distance;
pick sub-tour being served by chosen vehicle;
optimise sub-tour using forward SAT;
if *vehicle leaving depot* **then**
| spawn a new vehicle at depot;
end
implement the optimised sub-tour;

end

Finalise: Optimise vehicle tours with tightening SAT;

Algorithm 1: Running an episode of CVRP-TW. The same procedure is used during training and testing, with the exception that exploration steps are taken in a uniformly random fashion instead of the learnt RL policy.

Data: Customer locations, parameter $n \in \mathbb{Z}^+$

Result: Clusters, neighbourhood radius ρ

Compute Euclidean distances $d_{i,j}$ between customers;

Initialise set $\mathcal{K}$ of clusters as empty set;

while *at least one customer has no cluster mapping* **do**

Define a new empty cluster Φ ;
Add nearest unmapped customer from depot to Φ ;
for all *customers in Φ* **do**
| Add nearest n neighbours to Φ if these
| neighbours have no existing mapping;
| **if** *no new customers got added to Φ* **then**
| | break;
| **end**
end
Add cluster Φ to $\mathcal{K}$;

end

Set neighbourhood radius ρ as half of the largest cluster diameter in $\mathcal{K}$;

Finalise: Set of clusters $\mathcal{K}$ and radius ρ ;

Algorithm 2: Cluster preprocessing pseudo-code.

C. Reinforcement learning for routing

At each step in the rollout, each active vehicle-customer pair is evaluated independently, similar to the suggestion in prior work [19], [20]. The canonical reinforcement learning (RL) problem is specified by a Markov Decision Process $(S, \mathcal{A}, \mathcal{R}, T, \gamma)$, with S denoting the states, $\mathcal{A}$ denoting the actions, $\mathcal{R}$ denoting the rewards, T the transition probability $S \times \mathcal{A} \rightarrow S$, and $0 \leq \gamma \leq 1$ being a discount factor on future rewards [35]. In the present case, the states are defined with respect to a specified vehicle-customer pair in the context of all the customers in the data set. These features are listed in Table II. A neural network $Q(S, \mathcal{A}) : S \times \mathcal{A} \rightarrow \mathbb{R}$ is used

to estimate the *value* [35] of each feasible vehicle-customer pair (as defined by ability of vehicle to arrive at the customer before time window closure with sufficient capacity to meet customer demand).

The reward $\mathcal{R}$ associated with each decision is derived from the sequence of states as observed by each vehicle separately. The transition function T thus applies to trajectories of individual vehicles rather than to the episode as a whole (since different vehicles could move in successive RL steps, it does not make sense to trace the raw sequence of states observed in the episode). Let a vehicle k serves $p = \{1, \dots, P\}$ customers in its route, with each leg being of length d_p and requiring time t_p between time of service completion at the previous customer and the time of service start at the current customer. Then the reward for each decision is,

$$\mathcal{R}_{k,p} = \frac{\rho - d_p}{d_{max}} + \frac{\tau - t_p}{t_{max}} + \gamma^{P-p} \mathcal{R}_{term}, \text{ where} \quad (6)$$

$$\mathcal{R}_{term} = 2\rho - \frac{1}{P+1} \left(\sum_p d_p + D_{return} \right). \quad (7)$$

The reward defined in (6) provides an incentive for shorter legs in terms of both distance and time. The terminal reward $\mathcal{R}_{term}$ defined in (7) compares the average leg distance on a vehicle journey (including the final return leg D_{return} to the depot) to the largest cluster diameter 2ρ . The goal of training the neural network is to minimise the mean squared error between its output and the realised reward $\mathcal{R}_{k,p}$ for each decision.

The input is a vector of size 17, as specified in Table II. This is followed by a fully connected neural network with layers of size (128, 64, 32, 8) and an output layer of size 1 (scalar value). The hidden layers have $\tanh$ activation. A memory buffer of maximum $2^{16} = 65,536$ samples is maintained, with training happening after every 10 steps with a batch of size 4096 samples. The learning rate is 0.001. Exploration is implemented using an ϵ -greedy policy with ϵ decayed from 1 to 0 with a factor of 0.9995 after each episode. Exploration steps are taken uniformly randomly, while exploitation steps are chosen using a `softmax` function over the values attached to vehicle-customer pairs.

D. Policy rollouts

As explained in Sec II, the drawback of using vanilla RL is the unpredictable nature of the long-term effects, specifically the difficulty of estimating $\mathcal{R}_{term}$. Carrying out partial or complete rollouts using a trained policy is a popular recent improvement on Monte-Carlo Tree Search, and is called Neural-MCTS [4]. In this paper, the rollout is carried out up to episode completion for the top κ vehicle-customer pairs, as explained in Algorithm 1. At each time step, the learnt RL policy chooses a vehicle-customer pair based on a `softmax` function over the estimated values. Only feasible pairs are included, based on the ability of a given vehicle to serve a proposed customer within the time window and with sufficient demand, as defined by (3) and (5).

TABLE I
INPUTS FOR EVALUATING THE VALUE OF EACH VEHICLE-CUSTOMER PAIR,
WITH loc AS CURRENT LOCATION, c AS PROPOSED CUSTOMER, AND v AS
THE PROPOSED VEHICLE. ALL FEATURES ARE NORMALISED AS
EXPLAINED IN TEXT.

Input	Explanation
d	Distance from loc to c
$b_{d,short}$	Is d smaller than neighbourhood radius ρ
t	Time gap from now to start of service at c
$b_{t,short}$	Is time gap within time threshold τ
ngb	Are loc and c part of the same cluster
non_d	If $ngb = 1$, distance from c to nearest non-member
c_left	If $ngb = 0$, are any in-cluster customers left unserved
$drop_{far}$	If $c_left = 1$, are the dropped customers farther from depot than loc
$drop_{cls}$	If $c_left = 1$, are dropped customers within a distance of ρ from loc
$drop_{long}$	If $c_left = 1$, is the distance from dropped customers to nearest non-member more than distance from loc to dropped customers
$served$	How many customers of c cluster has v served so far
cls_{dem}	Could v serve all cluster members of c based on demand
$hops$	How many cluster members of c can be served before c
$clstim$	Is every cluster member feasible following c
$urgt$	How close to time window closure of c is v arriving
$dfrac$	Ratio of time being consumed for serving c to the fraction of v 's capacity being consumed
$remote$	How remote is the neighbourhood of c

Clearly, there are several other ways of neural MCTS such as partial episodic rollout followed by estimation of residual value. One may also experiment with different values of κ , which is described in Sec. V. Finally, one may perform multiple rollouts in order to maximise the probability of finding good solutions, along similar lines as most meta-heuristics. A specific choice about the breadth, depth, and repetitions of rollout can be made based on the problem at hand and the available computational resources.

E. Satisfiability solver formulation

The solver chosen for optimisation of sub-problems in this work is Z3 [36]. Given the underlying nonlinear nature of the problem as described in (1)-(5), the typical mixed-integer linear program for CVRP-TW uses big-M constraints [37] to handle the conditional statements. However, this approach has well-known computational challenges. A satisfiability (SAT) solver, on the other hand, provides a way of naturally specifying logical constraints, and modern SAT solvers are known to be both efficient and able to handle optimisation objectives through the ‘MAX-SAT’ approach [38].

Experiments with different ways of formulating the MAX-SAT problem for CVRP-TW showed that Z3 could handle strongly nonlinear constraints, but struggled with a larger number of decision variables. This appears to be in clear contrast with mixed-integer program solvers, where simplicity of constraints is prioritised over the number of variables. Based on this intuition, I eliminated the connectivity variables $a_{j,i,k}$ (even with a fixed vehicle k , the number of variables increases as $|C|^2$) as well as the first/last indicators $f_{i,k}$ and $l_{i,k}$. In lieu of these variables, we shall specify the route of the vehicle using an integer order variable $O_{i,k} \in \{1, 2, \dots, |C|\}$.

We also take inspiration from work on constrained position shifting (CPS) from air traffic management [39] to improve the search efficiency of the SAT solver. Intuitively, CPS allows for a maximum deviation in the planned order (compared to the route generated by rollout) up to a maximum limit of Δ . This parameter is successively tightened until an optimal solution is produced in the set time limit. The problem with $\Delta = 0$ has a trivial solution identical to the rollout route, although opportunistic customers may be added to this as explained below. We consider two types of MAX-SAT problems. The first formulation (*forward SAT*) is used while the routes are being computed, in order to optimise the rollout outputs. The second solution (*tightening SAT*) is used to tighten the final outputs after the episode is completed.

1) *Forward SAT*: In the following description, we may omit the vehicle subscript k as the formulation optimises the proposed route for a specified vehicle only. Assume that a sub-tour R is generated by the rollout policy as per Algorithm 1, consisting of a sequence $R_i \forall$ customers $c_i \in R$. We also define a set A of additional customers which are active and which are cluster neighbours of the members of R .

We wish to compute optimised integer ordering variables O_i and service start times T_i for customers in $R \cup A$. The forward-SAT formulation is given by,

$$J = \min_{O_i} \left[\left(\sum_{i \in R \cup A} D_i \right) + D_{return} \right], \text{ where} \quad (8)$$

$$D_i = \begin{cases} d_{loc,i} & \text{if } O_i = 1 \\ d_{j,i} & \text{if } O_i > 1 \text{ and } O_i = O_j + 1 \\ d_{max} & \text{if } O_i = -1 \text{ i.e. } c_i \text{ not served} \end{cases} \quad (9)$$

$$D_{return} = d_{o,i} \text{ for last } c_i \text{ being served} \quad (10)$$

The variable D_i represents the leg distance to get to customer c_i . For the first customer on the sub-tour, the distance is computed from the current location loc (see Table II). For all other legs, it is the distance between successive customer locations. We will assign a distance penalty d_{max} as defined in Sec. IV-B for any customers that are skipped. Finally, D_{return} is the distance to return to the depot, should the vehicle be unable to serve any further customers. Including this term encourages closed-loop journeys over ones which could end very far from the depot.

Note that J in (8) is equivalent to the one defined in (1), but is only reformulated in terms of ordering variables O_i . Similarly, we can easily formulate the system constraints (2), (3), (4), and (5) with the understanding that $(f_i = 1) \equiv (O_i = 1)$ and $(a_{i,j} = 1) \equiv (O_j = O_i + 1)$.

As formulated above, the solver is able to generate solutions within 1 second for small problems (sub-tours consisting of up to 6 customers). To further improve the computational time, we trade off some optimality by introducing constrained position shifting [39]. The CPS constraints are given by,

$$(R_i - \Delta \leq O_i \leq R_i + \Delta) \forall c_i \in R \quad (11)$$

$$O_i \geq 1 \forall c_i \in R \quad (12)$$

Constraint (11) restricts the deviation in order for the customers that are already part of the sub-tour R . Note that the additional customers in A are not included. The final constraint (12) forces the solver to serve all customers already in R .

2) *Tightening SAT*: The MAX-SAT formulation described in (8)-(12) can be reused for tightening the final routes after episode completion, with minor changes. The key change is that we only consider the tour R , without any additional opportunistic customers. We also start with a larger value of CPS parameter Δ , to explore potentially greater optimisation opportunities. Finally, we note that the tour must start and end at the depot, which implies that $loc \equiv o$.

V. RESULTS AND DISCUSSION

The RL algorithm described in Sec. IV-C is trained on all instances of the Solomon data set [2] containing 25 customers (a total of 56 data sets). The results of training over 6000 episodes for 3 random seeds is shown in Fig. 1, with the hyperparameters as described in Sec. IV-C. The MAX-SAT formulation is only activated for the final 2000 episodes, with the first 4000 episodes proceeding with RL and rollouts only. A steady trend is seen in both the rewards received and the moving average of distance over episodes.

The testing of the learnt model is carried out on all 56 data sets in the Solomon data [2] for 25, 50, and 100 customers. There are three types of geographical distributions: C stands for clear clusters, R stands for a random distribution, while RC stands for a mix of clustered and random. Additionally, data sets tagged with ‘1’ (as in RC1) have low vehicle capacity, while ones tagged with 2 have higher vehicle capacity. The first of these is the same set that is used for training, while the latter two are unseen. The results are compiled in Table II. The first column in the table is the average best-known distance in literature. The next two columns GA and RH are values reported in [20], while QL are the values reported in [19]. The final two columns are from the current proposal, with RL+RO including only rollouts while RL+SAT+RO also includes the MAX-SAT formulation.

Table II also reports the average computation time at the start of each set of data, wherever available. A comparison of the distances and computation times shows a clear tradeoff between solution quality (lower distances are better) and computation times. Note that in a few instances, RL+SAT+RO outperforms GA on both route distance and computation time. In one type of distribution (C2), the proposed method appears to result in somewhat poor performance. A deeper look at the generated routes in this case reveals that the learnt RL policy does not foresee the possibility of a single vehicle with high capacity serving multiple clusters in the data. This is a discrepancy which will be addressed in future work.

Finally, the tradeoff in computation time and solution quality is made clear by the experiment reported in Fig. 2, where we see the effect of varying κ (number of rollout branches) on computation time, number of nodes evaluated, and the optimality gap of the resulting solution. We see that as the number of rollouts increase, the computation time increases

TABLE II
COMPARISON OF RESULTS, WITH NUMBERS REPRESENTING AVERAGE DISTANCE OF THE ROUTES IN A GIVEN TYPE OF DATA. APPROXIMATE COMPUTATION TIMES ARE LISTED AT THE TOP OF EACH COLUMN. QUESTION MARKS INDICATE VALUES WHICH ARE NOT AVAILABLE IN LITERATURE. THE FIRST COLUMN IS THE BEST KNOWN DISTANCE COMPILED FROM MULTIPLE SOURCES [2]. METHODS GA AND RH ARE FROM [20], WHILE QL IS FROM [19]. THE FINAL TWO COLUMNS ARE FROM THE CURRENT APPROACH, WITH $RL + RO$ USING ROLLOUTS ONLY, AND $RL + SAT + RO$ ALSO INCLUDING THE SAT SOLVER.

Data	<i>Best dist</i>	<i>GA</i>	<i>RH</i>	<i>QL</i>	<i>RL+ RO</i>	<i>RL+ SAT+ RO</i>
	(8400s)	(35s)	(1s)	(5s)	(8s)	(15s)
C1-25	191	191	283	215	199	192
R1-25	464	469	619	586	620	568
RC1-25	350	351	380	489	395	362
C2-25	216	245	297	246	273	261
R2-25	382	420	540	521	563	513
RC2-25	319	365	598	448	352	329
	(??)	(150s)	(2s)	(30s)	(25s)	(60s)
C1-50	362	362	676	407	398	374
R1-50	766	785	1114	982	1067	963
RC1-50	730	741	1011	951	963	849
C2-50	357	401	775	457	569	525
R2-50	634	652	989	891	878	824
RC2-50	585	621	1203	811	673	631
	(??)	(1200s)	(??)	(90s)	(150s)	(350s)
C1-100	826	830	(??)	975	914	869
R1-100	1210	1210	(??)	1620	1790	1646
RC1-100	1384	1385	(??)	1800	1977	1788
C2-100	587	589	(??)	787	1105	1012
R2-100	902	902	(??)	1341	1428	1309
RC2-100	1063	1063	(??)	1497	1548	1427

steadily while the solution quality improves. This provides justification for the claim that the proposed RL+RO+SAT approach allows us to choose from a spectrum of policies from pure RL (towards lower value of κ) to full meta-heuristics (high values of κ , with multiple rollouts in each branch).

In conclusion, this paper bridges the gap between three extreme approaches for CVRP-TW, namely heuristics (high design time), meta-heuristic (high computation time), and reinforcement learning (poor solution quality). Future work in this area will attempt to formalise the counterfactual objective of setting the optimal hyperparameters of RL+RO+SAT given a target for computation time and solution quality.

REFERENCES

- [1] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problems with time window constraints,” *Operations research*, vol. 35, no. 2, pp. 254–265, 1987.
- [2] SINTEF, “Solomon benchmark,” Retrieved May 2022. [Online]. Available: <https://www.sintef.no/projectweb/top/vrptw/solomon-benchmark/>
- [3] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, “A survey of monte carlo tree search methods,” *IEEE Transactions on Computational Intelligence and AI in games*, vol. 4, no. 1, pp. 1–43, 2012.
- [4] T. Anthony, Z. Tian, and D. Barber, “Thinking fast and slow with deep learning and tree search,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] M. L. Fisher and R. Jaikumar, “A generalized assignment heuristic for vehicle routing,” *Networks*, vol. 11, no. 2, pp. 109–124, 1981.
- [6] D. Pisinger and S. Ropke, “A general heuristic for vehicle routing problems,” *Computers & operations research*, vol. 34, no. 8, pp. 2403–2435, 2007.

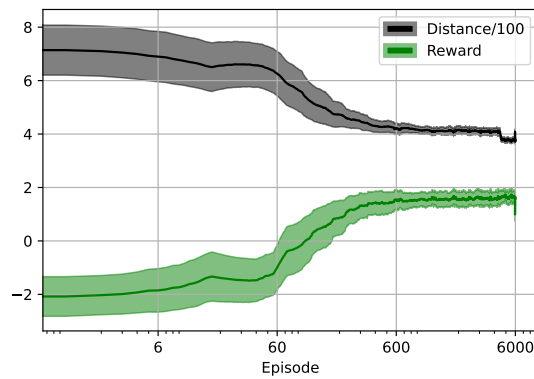


Fig. 1. Training results over three random seeds, using the Solomon 25-customer data sets. Note that the x-axis is logarithmic.

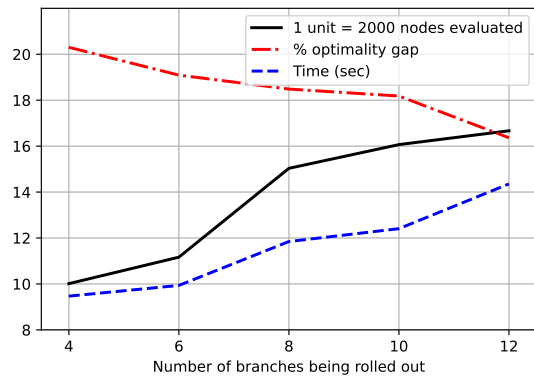


Fig. 2. Tradeoff between rollout extent, solution quality, computation time.

[7] A. M. Campbell and M. Savelsbergh, "Efficient insertion heuristics for vehicle routing and scheduling problems," *Transportation science*, vol. 38, no. 3, pp. 369–378, 2004.

[8] B.-L. Garcia, J.-Y. Potvin, and J.-M. Rousseau, "A parallel implementation of the tabu search heuristic for vehicle routing problems with time window constraints," *Computers & Operations Research*, vol. 21, no. 9, pp. 1025–1033, 1994.

[9] P. S. Barma, J. Dutta, and A. Mukherjee, "A 2-opt guided discrete antlion optimization algorithm for multi-depot vehicle routing problem," *Decision Making: Applications in Management and Engineering*, vol. 2, no. 2, pp. 112–125, 2019.

[10] N. Kohl, J. Desrosiers, O. B. Madsen, M. M. Solomon, and F. Soumis, "2-path cuts for the vehicle routing problem with time windows," *Transportation Science*, vol. 33, no. 1, pp. 101–116, 1999.

[11] A. Larsen, O. B. Madsen, and M. M. Solomon, "Recent developments in dynamic vehicle routing systems," in *The vehicle routing problem: Latest advances and new challenges*. Springer, 2008, pp. 199–218.

[12] O. Bräysy and M. Gendreau, "Vehicle routing problem with time windows, part ii: Metaheuristics," *Transportation science*, vol. 39, no. 1, pp. 119–139, 2005.

[13] M. Gendreau, J.-Y. Potvin, O. Bräumlaysy, G. Hasle, and A. Løkketangen, "Metaheuristics for the vehicle routing problem and its extensions: A categorized bibliography," in *The vehicle routing problem: latest advances and new challenges*. Springer, 2008, pp. 143–169.

[14] W.-C. Chiang and R. A. Russell, "A reactive tabu search metaheuristic for the vehicle routing problem with time windows," *INFORMS Journal on computing*, vol. 9, no. 4, pp. 417–430, 1997.

[15] B. D. Backer, V. Furnon, P. Shaw, P. Kilby, and P. Prosser, "Solving vehicle routing problems using constraint programming and metaheuristics," *Journal of heuristics*, vol. 6, no. 4, pp. 501–523, 2000.

[16] B. M. Baker and M. Ayechev, "A genetic algorithm for the vehicle

routing problem," *Computers & Operations Research*, vol. 30, no. 5, pp. 787–800, 2003.

[17] F. T. Hanshar and B. M. Ombuki-Berman, "Dynamic vehicle routing using genetic algorithms," *Applied Intelligence*, vol. 27, no. 1, pp. 89–99, 2007.

[18] J. E. Bell and P. R. McMullen, "Ant colony optimization techniques for the vehicle routing problem," *Advanced engineering informatics*, vol. 18, no. 1, pp. 41–48, 2004.

[19] N. N. Sultana, V. Baniwal, A. Basumatary, P. Mittal, S. Ghosh, and H. Khadilkar, "Fast approximate solutions using reinforcement learning for dynamic capacitated vehicle routing with time windows," *arXiv preprint arXiv:2102.12088*, 2021.

[20] A. Gupta, S. Ghosh, and A. Dhara, "Deep reinforcement learning algorithm for fast solutions to vehicle routing problem with time-windows," in *Joint International Conference on Data Science & Management of Data*, 2022, pp. 236–240.

[21] C. Malandraki and M. S. Daskin, "Time dependent vehicle routing problems: Formulations, properties and heuristic algorithms," *Transportation science*, vol. 26, no. 3, pp. 185–200, 1992.

[22] B. Kallehauge, "Formulations and exact algorithms for the vehicle routing problem with time windows," *Computers & Operations Research*, vol. 35, no. 7, pp. 2307–2330, 2008.

[23] A. Agra, M. Christiansen, R. Figueiredo, L. M. Hvattum, M. Poss, and C. Requejo, "The robust vehicle routing problem with time windows," *Computers & operations research*, vol. 40, no. 3, pp. 856–866, 2013.

[24] Google, "OR-Tools." [Online]. Available: <https://developers.google.com/optimization>

[25] H. C. Lau, M. Sim, and K. M. Teo, "Vehicle routing problem with time windows and a limited number of vehicles," *European journal of operational research*, vol. 148, no. 3, pp. 559–569, 2003.

[26] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," *Advances in neural information processing systems*, vol. 28, 2015.

[27] Q. Ma, S. Ge, D. He, D. Thaker, and I. Drori, "Combinatorial optimization by graph pointer networks and hierarchical reinforcement learning," *arXiv preprint arXiv:1911.04936*, 2019.

[28] I. Drori, A. Kharkar, W. R. Sickinger, B. Kates, Q. Ma, S. Ge, E. Dolev, B. Dietrich, D. P. Williamson, and M. Udell, "Learning to solve combinatorial optimization problems on real-world graphs in linear time," in *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2020, pp. 19–24.

[29] Q. Cappart, D. Chételat, E. Khalil, A. Lodi, C. Morris, and P. Veličković, "Combinatorial optimization and reasoning with graph neural networks," *arXiv preprint arXiv:2102.09544*, 2021.

[30] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takác, "Reinforcement learning for solving the vehicle routing problem," *Advances in neural information processing systems*, vol. 31, 2018.

[31] J. James, W. Yu, and J. Gu, "Online vehicle routing with neural combinatorial optimization and deep reinforcement learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 10, pp. 3806–3817, 2019.

[32] A. Delarue, R. Anderson, and C. Tjandraatmadja, "Reinforcement learning with combinatorial actions: An application to vehicle routing," *Advances in Neural Information Processing Systems*, vol. 33, pp. 609–620, 2020.

[33] K. Zhang, F. He, Z. Zhang, X. Lin, and M. Li, "Multi-vehicle routing problems with soft time windows: A multi-agent reinforcement learning approach," *Transportation Research Part C: Emerging Technologies*, vol. 121, p. 102861, 2020.

[34] A. Likas, N. Vlassis, and J. J. Verbeek, "The global k-means clustering algorithm," *Pattern recognition*, vol. 36, no. 2, pp. 451–461, 2003.

[35] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[36] L. d. Moura and N. Bjørner, "Z3: An efficient smt solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.

[37] R. Raman and I. E. Grossmann, "Modelling and computational techniques for logic based integer programming," *Computers & Chemical Engineering*, vol. 18, no. 7, pp. 563–578, 1994.

[38] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided maxsat resolution," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, no. 1, 2014.

[39] H. Balakrishnan and B. Chandran, "Scheduling aircraft landings under constrained position shifting," in *AIAA guidance, navigation, and control conference and exhibit*, 2006, p. 6320.